

Daily Scholar Papers Report — 2026-08-14

2026-08-14

Daily Scholar Papers Report — 2026-08-14

[Download PDF](#)

Window covered: 2026-08-13 → 2026-08-14 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

One paper carries forward from today’s window, and the more consequential result is that the window was not empty at all.

Two candidates arrived, both via the Scholar recommendation channel; the user-curated self-email queue was silent. One candidate cleared Stage-1 screening and one did not. The paper that cleared is **FlowArk**, a preprint on agentic data-flow analysis for Android apps, which sits at the intersection of classical source-to-sink taint analysis and the use of LLM coding agents as analysis engines rather than as code generators — a junction this digest has been tracking for several months.

The deep-read of that paper is **incomplete, and the report says so rather than papering over it**. The paper’s full text could not be obtained this run: the local PDF library was unreachable, the arXiv page returned a rate-limit error on both attempts, and a search fallback surfaced no indexed record of the work. What appears below is derived entirely from the abstract fragment carried in the alert itself, which Scholar truncates mid-sentence — including, unhelpfully, the sentence that states the limitation FlowArk exists to fix. The paper is classified **Keep** on that basis. This is a statement about the evidence in hand, not about the work’s quality; on premise alone it is the most interesting item in the window, and it is queued for a full re-read on the next healthy run.

The second result concerns the channel rather than any paper. The 2026-08-03 run closed six consecutive zero-intake days by escalating, and named a cancelled or silently-disabled alert subscription as the leading hypothesis — while noting that the question could not be settled from inside the mailbox. Today’s delivery settles it. **The subscription is alive**. The eleven-day gap was sparse recommendation output at the source, not a broken pipe. No configuration change is needed.

Outstanding: 0 · Keep: 1 · Borderline High-Priority: 0

Highlighted Papers

Title	Authors	Venue	Link
FlowArk: Boosting Agentic Data-flow Analysis for	Y. Zhang, J. Wu, Y. Nan	arXiv preprint, 2026	arXiv:2607.11308

Title	Authors	Venue	Link
Android Apps via Context-Aware Knowledge Reuse			

Papers

1.1 · PROGRAM ANALYSIS · FlowArk proposes context-aware knowledge reuse to keep coding agents from re-deriving the same Android data-flow facts 🙌🤔👉

FlowArk: Boosting Agentic Data-flow Analysis for Android Apps via Context-Aware Knowledge Reuse Y. Zhang, J. Wu, Y. Nan — arXiv preprint arXiv:2607.11308, 2026 <https://arxiv.org/abs/2607.11308>

Scope note — read this first

This entry is **abstract-only**. The paper’s full text could not be retrieved during this run, and the abstract as delivered by the alert is itself cut off mid-sentence. Everything below is either stated in that fragment or explicitly marked as unestablished. No numbers, no algorithm, no evaluation claims are reported, because none were seen. A full deep-read is queued for the next run.

What the paper sets out to do

The premise is that data-flow analysis underpins privacy and security auditing of Android apps, and that recent LLM coding agents can already take on non-trivial source-to-sink data-flow tasks by searching, reading, and reasoning over repository code. That framing is worth pausing on, because it treats the agent as a *program-analysis instrument* rather than as a code-writing assistant, and it accepts the agent’s baseline competence as given rather than as the thing to be demonstrated. The paper’s contribution begins after that concession — it is about what goes wrong next.

What goes wrong is where the record stops. The abstract turns to the limitation with “However, when ...” and the excerpt ends there. **The limitation itself is not recoverable from the material available**, and it is not guessed at here.

What can be read off the title is the shape of the proposed remedy: an artefact named **FlowArk**, built around **context-aware knowledge reuse**. Read literally, that names a strategy of retaining analysis facts derived in one context and re-applying them in another, rather than having the agent re-derive them from source on each query — with “context-aware” governing when a retained fact is legitimately transferable. Whether that reading matches the paper’s actual mechanism is unverified.

Why it is worth a full read

Two reasons, both independent of the specific results.

The first is that the problem it targets, if the reading above is right, is not an Android problem. Any agentic analysis that re-reads and re-reasons over the same repository for every query pays the same cost repeatedly, and the mismatch between per-query context budgets and whole-program facts is the central scaling obstacle for this entire class of tool. A durable answer would transfer to other languages and other analyses, which is exactly the methodology-reusability criterion this digest screens on.

The second is that Android source-to-sink analysis is an unusually honest testbed for it. The sources and sinks are enumerable, the ground truth is comparatively tractable, and there is a deep bench of classical static-analysis baselines to measure against — so a claimed improvement can be checked rather than merely asserted. Agentic-analysis papers frequently pick domains where that is not true.

Not established

For the record, and to keep the next run honest about what it still owes: the mechanism of knowledge reuse; what “context-aware” ranges over; the evaluation benchmark, subjects, and baselines; every quantitative result; whether the approach is agent-agnostic or tied to a particular model; and the paper’s licence. No figures are embedded, as licence status is undetermined and the embedding rules require a CC-class licence. No formal notation appears, as none was available from the paper’s own text.

Cross-Paper Synthesis

A one-paper window does not support cross-paper synthesis in the usual sense, so the more useful comparison is against the archive.

FlowArk is the latest instance of a pattern that has been recurring in this digest through 2026: the LLM agent arriving not as the thing being analysed, nor as a code generator, but as a *component inside a program-analysis pipeline* — and the resulting papers converging on the same bottleneck from different directions. Whether the target is taint-style vulnerability detection, source-to-sink privacy flows, or interprocedural reasoning at large, the wall is the same one. Whole-program facts do not fit in a per-query context budget, and an agent that re-establishes them from source on every question spends most of its budget rediscovering what it already knew.

The interesting divergence is in the proposed escapes. Some work pushes toward better retrieval — find the right slice of the repository for this query. Some pushes toward decomposition — break the flow into sub-queries each of which fits. FlowArk’s title points at a third option: **amortisation**, where the fix is not to shrink the question but to stop re-answering the parts already settled. These are not mutually exclusive, and a reasonable prediction is that the durable systems combine them. Whether FlowArk is in fact an instance of the third strategy is exactly what a full read needs to confirm, and the confirmation is not yet in hand.

Writing & Rationale Insights

Today’s run produced one point about reporting under degraded conditions and one about how a monitoring pipeline should close out an escalation.

On degraded reporting: the classification should track the evidence, not the impression. FlowArk is the most promising item to reach this digest in over a week, and the material available on it amounts to four sentences, one of which is cut off. There is a real pull, in that situation, to write the entry the paper *deserves* — to fill the gap from domain knowledge, infer the mechanism from the title, and produce something that reads like a proper deep-read. That would have been the wrong output, and not only because the inferences might be wrong. It would have been wrong because a reader cannot tell a confident inference from a verified fact once both are written in the same voice, and a digest whose entries are not distinguishable on that axis is not usable as a filter. The discipline that actually helps is

cheap: state the access failure at the top of the entry rather than in a footnote, mark the inferences as inferences where they appear, and enumerate what remains unestablished so the next run inherits a checklist rather than a false sense of completion. The classification then follows mechanically — Outstanding requires verbatim definitions, algorithms, and numbers; none were obtainable; the paper is Keep. Keeping the ranking honest about *evidence* rather than about *promise* is what lets promise be recorded separately and loudly, as it is above.

On closing an escalation: let the world answer, and then say that it did. The 2026-08-03 report crossed a threshold it had set in advance, named a hypothesis — cancelled or disabled subscription — and stated plainly that the question could not be resolved from inside the mailbox. Today it resolved itself: the alert arrived, and the hypothesis is dead. Two things about that are worth generalising. The first is that the escalation was still correct despite being wrong, because a threshold written in advance and acted on is what converts a run of null results into a decision, and the alternative — another round of increasingly elaborate verification that the emptiness is real — would have produced more confidence and no new information. The second is that the resolution deserves the same prominence the escalation got. There is an asymmetry in how monitoring systems report: alarms are loud and all-clears are silent, which over time trains the reader to discount the alarms. Recording “the hypothesis raised on 08-03 is falsified, the subscription is healthy, no action needed” costs one sentence and is the thing that keeps the next escalation worth reading.